

UCSD Machine Learning Bootcamp Capstone Deployment Writeup

Kristian Eberhardson

Using a Large Language Model (LLM), I created an Angular app that tutors its users on the Swedish language for my capstone project. The overall project consists of three major components: the GitHub repo of my app's source code, an instance of my app launched via the AWS Amplify service, and the Mistral Large 3 LLM in AWS Bedrock.

AWS Amplify simplifies the deployment of full-stack apps made with JavaScript-based frameworks, such as Angular (I have five years of professional experience in Angular). It requires only my GitHub repo and my consent in order to produce a publicly accessible link. AWS Amplify automatically locates the server on which my app runs and launches it on my behalf with the click of a button. Every time I push a code change to the GitHub repo, AWS Amplify automatically updates the deployed app. AWS Amplify charges me a nominal fee for the duration in which the app stays up. I can take the app down at any time. The only catch is that if I wish to test my own code on my laptop, I need to deploy the app first.

AWS Amplify integrates seamlessly with the other AWS services. When I deploy the app, AWS Amplify automatically creates an AWS Lambda function that sends and receives the prompts in my code from an LLM of my choice in AWS Bedrock whenever that code is run. For this capstone, I have chosen the Mistral Large 3 LLM, which supports 675 billion parameters and is trained in both English and Swedish. [1, 2] My block of code assigns Mistral Large 3 the following system role: "You are my Swedish tutor. You ask me questions in Swedish and accept only Swedish responses. For each incorrect word in my response, you explain the concept associated with the incorrect word and provide the correct word." For the deployed app, I can modify the parameters of the underlying Lambda function, such as the timeout it is willing to wait for a response before returning an error. This is important because the default timeout for a Lambda function is three seconds, whereas sending and receiving a prompt from Mistral takes roughly five seconds per prompt in the app.

AWS provides the CloudWatch service to monitor apps deployed via AWS Amplify. A user may use AWS CloudWatch to attach one or more alarms to an AWS Amplify app. [2] The alarm watches for a list of supported metrics, such as latency, i.e. the time it takes for the network to respond to a command made by a user in the app. [3] For example, I can set the alarm to trigger if the latency exceeds a certain number of milliseconds and send an email to me whenever the app is too slow. [3] Other such metrics include watching for HTTP status code errors in the 400-600 range. [3] AWS Amplify automatically stores access logs for all user requests made in the app; I need only select the two-week period for which I wish to obtain access logs. [4] The corresponding access logs remain available until I take the app down.

Works Referenced

[1] "Introducing Mistral 3". <https://mistral.ai/news/mistral-3>

[2] "Your First Text Prompts". <https://help.mistral.ai/en/articles/424379-your-first-text-prompts>

[3] "Monitoring an Amplify Application with AWS CloudWatch".
<https://docs.aws.amazon.com/amplify/latest/userguide/monitoring-with-cloudwatch.html>

[4] "Retrieving and Analyzing Access Logs for an Amplify Application".
<https://docs.aws.amazon.com/amplify/latest/userguide/using-access-logs.html>